

Homework 3: [YOUR NAME HERE]

Introduction to Time Series, Fall 2024

2001-10-11

The total number of points possible for this homework is 38. The number of points for each question is written below, and questions marked as “bonus” are optional (points awarded for bonus problems can be used to earn back points that you may have lost on other parts of this homework but will not put you above full credit). Submit the **knitted pdf file** from this Quarto document to Gradescope.

Regression troubles

1. (5 pts) Suppose that $y \in \mathbb{R}^n$ is a response vector and $X \in \mathbb{R}^{n \times p}$ is a predictor matrix, with $p > n$. Prove that there is at least one $\eta \neq 0$ (not equal to the zero vector) that is in $\text{null}(X)$, the null space of X . Prove that if $\tilde{\beta}$ is a least squares solution in the regression of y on X , then any vector of the form

$$\hat{\beta} = \tilde{\beta} + \eta, \quad \text{for } \eta \in \text{null}(X)$$

is also a solution.

Proof.

(Type your proof here)

2. (6 pts) With X, y as in Q1, suppose that $\tilde{\beta}$ is a least squares solution with $\tilde{\beta}_j > 0$, and suppose that $\text{null}(X) \not\perp e_j$, where e_j is the j^{th} standard basis vector (i.e., e_j is a vector with all 0s except for a 1 in the j^{th} component), and recall we write $S \perp v$ for a set S and vector v provided $u^T v = 0$ for all $u \in S$. Prove that there exists another least squares solution $\hat{\beta}$ such that $\hat{\beta}_j < 0$.

Proof.

(Type your proof here)

Ridge and lasso

3. (8 pts) Using the cardiovascular mortality regression data, form lagged features from the particulate matter and temperature variables, using lags 4, 8, ..., 40 from each. Using the `glmnet` package, fit a ridge regression and lasso regression (two separate models), each over a grid of tuning parameter values λ chosen by the `glmnet()` function, with cardiovascular mortality as the response and all the lagged features as predictors (you should have 20 in total: 10 from particulate matter, and 10 from temperature). However, make sure you do this in a split-sample setup for validation, as follows, for each of ridge and lasso:

- fit the model on the *first half of the time series*;
- make predictions on the *second half of the time series*, for each λ ;
- record the MAE of the predictions on the second half, for each λ ;
- choose and report the value of λ with the lowest MAE;
- plot the cardiovascular mortality time series, along with the predictions on the second half, and print the MAE and the selected value of λ on the plot.

You can build on the code from the regularization lecture for fitting the ridge and lasso models, and the regression lecture for the split-sample validation. *Note carefully that the lecture code includes lag 0, and here we do not, so that we can make ex-ante 4-week ahead forecasts!*

```
# Create lagged features
lags = list(range(4, 40, 4))
df = pd.DataFrame({"mort": cmort["value"]})
for lag in lags:
    df[f"part_lag{lag}"] = part["value"].shift(lag)
    df[f"temp_lag{lag}"] = tempr["value"].shift(lag)

df = df.dropna().reset_index(drop=True)
n = len(df)
n_half = n // 2

# Train-test split
train_df = df.iloc[:n_half]
test_df = df.iloc[n_half:]

X_train = train_df.drop(columns=["mort"])
y_train = train_df["mort"]
X_test = test_df.drop(columns=["mort"])
```

```

y_test = test_df["mort"]

# Standardize predictors using training set scaling
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# glmnet-style grid anchored at alpha_max (smallest lambda that zeroes all lasso coefs)
alpha_max = np.max(np.abs(X_train_scaled.T @ y_train)) / len(y_train)
lambdas_lasso = np.logspace(np.log10(alpha_max * 1e-4), np.log10(alpha_max), 200)
lambdas_ridge = np.logspace(np.log10(alpha_max * 1e-4), np.log10(alpha_max * 1e4), 200)

# Lasso
lasso_maes = []
for lam in lambdas_lasso:
    model = Lasso(alpha=lam, max_iter=10000)
    model.fit(X_train_scaled, y_train)
    preds = model.predict(X_test_scaled)
    lasso_maes.append(mean_absolute_error(y_test, preds))

best_idx_l = np.argmin(lasso_maes)
best_lambda_l = lambdas_lasso[best_idx_l]
best_mae_l = lasso_maes[best_idx_l]

# Optimal Lasso predictions
best_lasso = Lasso(alpha=best_lambda_l, max_iter=10000)
best_lasso.fit(X_train_scaled, y_train)
lasso_preds = best_lasso.predict(X_test_scaled)

# Ridge
ridge_maes = []
for lam in lambdas_ridge:
    model = Ridge(alpha=lam)
    model.fit(X_train_scaled, y_train)
    preds = model.predict(X_test_scaled)
    ridge_maes.append(mean_absolute_error(y_test, preds))

best_idx_r = np.argmin(ridge_maes)
best_lambda_r = lambdas_ridge[best_idx_r]
best_mae_r = ridge_maes[best_idx_r]

# Optimal Ridge predictions

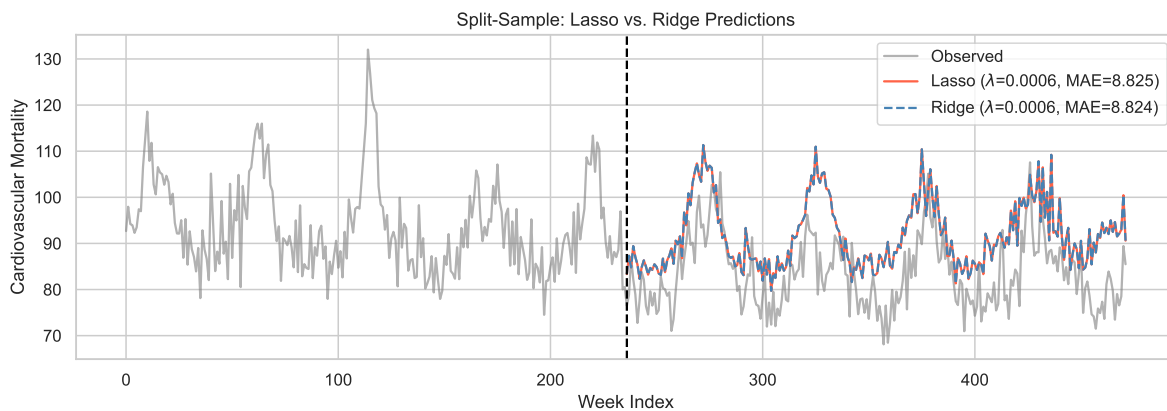
```

```

best_ridge = Ridge(alpha=best_lambda_r)
best_ridge.fit(X_train_scaled, y_train)
ridge_preds = best_ridge.predict(X_test_scaled)

# Plot Lasso and Ridge split-sample results
fig, ax = plt.subplots(figsize=(11, 4))
sns.lineplot(x=df.index, y=df["mort"], color="grey", alpha=0.6, ax=ax, label="Observed")
sns.lineplot(x=test_df.index, y=lasso_preds, color="tomato", ax=ax, label=f"Lasso ( $\lambda=0.0006$ , MAE=8.825)")
sns.lineplot(x=test_df.index, y=ridge_preds, linestyle="--", color="steelblue", ax=ax, label=f"Ridge ( $\lambda=0.0006$ , MAE=8.824)")
ax.axvline(x=n_half, color="black", linestyle="--")
ax.set_title("Split-Sample: Lasso vs. Ridge Predictions")
ax.set_xlabel("Week Index")
ax.set_ylabel("Cardiovascular Mortality")
ax.legend()
plt.tight_layout()
plt.show()

```



4. (1 pts) Which lagged features were present in the MAE-optimal lasso model, selected by split-sample validation, in Q3?

```

# Lasso coeffs
active_features = [X_train.columns[i] for i in range(len(best_lasso.coef_)) if best_lasso.coef_[i] != 0]
print("Active features in the optimal Lasso model:")
for feat in active_features:
    coef_val = best_lasso.coef_[X_train.columns.get_loc(feat)]
    print(f" {feat}: coefficient = {coef_val:.4f}")

#Ridge coeffs
active_features = [X_train.columns[i] for i in range(len(best_ridge.coef_)) if best_ridge.coef_[i] != 0]

```

```

print("Active features in the optimal Ridge model:")
for feat in active_features:
    coef_val = best_ridge.coef_[X_train.columns.get_loc(feat)]
    print(f" {feat}: coefficient = {coef_val:.4f}")

```

Active features in the optimal Lasso model:

```

part_lag4: coefficient = 3.8813
temp_lag4: coefficient = -2.4325
part_lag8: coefficient = 2.9592
temp_lag8: coefficient = -1.5241
part_lag12: coefficient = 0.8779
temp_lag12: coefficient = 0.5463
part_lag16: coefficient = -0.3399
temp_lag16: coefficient = 0.0504
part_lag20: coefficient = -2.0916
temp_lag20: coefficient = 2.3615
part_lag24: coefficient = 0.4444
temp_lag24: coefficient = -0.6993
part_lag28: coefficient = 1.2239
temp_lag28: coefficient = -1.1295
part_lag32: coefficient = 0.2554
temp_lag32: coefficient = -0.3608
part_lag36: coefficient = -0.2064
temp_lag36: coefficient = -0.2173

```

Active features in the optimal Ridge model:

```

part_lag4: coefficient = 3.8827
temp_lag4: coefficient = -2.4344
part_lag8: coefficient = 2.9623
temp_lag8: coefficient = -1.5263
part_lag12: coefficient = 0.8807
temp_lag12: coefficient = 0.5463
part_lag16: coefficient = -0.3421
temp_lag16: coefficient = 0.0524
part_lag20: coefficient = -2.0958
temp_lag20: coefficient = 2.3642
part_lag24: coefficient = 0.4480
temp_lag24: coefficient = -0.7047
part_lag28: coefficient = 1.2256
temp_lag28: coefficient = -1.1312
part_lag32: coefficient = 0.2576
temp_lag32: coefficient = -0.3624
part_lag36: coefficient = -0.2080

```

```
temp_lag36: coefficient = -0.2165
```

(Type your discussion here)

5. (8 pts) Repeat Q3, except implement time series cross-validation instead of split-sample validation. You should begin time series cross-validation on the second half of the time series, treating the first half as a burn-in set. Also, be sure to fit each ridge or lasso model using a trailing window of 200 time points (not all past).

Note that in Python, we pass explicit `lambda` parameters to scikit-learn models ahead of time, which naturally circumvents the variable tuning grid issues seen in R. We can use the same `lambdas_lasso` and `lambdas_ridge` grids defined in Q3. Produce plots of the cardiovascular mortality time series, along with the predictions from time series CV on the second half, and print the MAE and the selected value of λ on the plots.

```
tscv = TimeSeriesSplit(n_splits=5)

lasso_cv_preds = []
ridge_cv_preds = []
lasso_alphas_selected = []
ridge_alphas_selected = []
lasso_coefs_history = []
y_cv_true = []

# Rolling TS CV loop using LassoCV / RidgeCV for adaptive lambda selection
for t in range(n_half, n):
    window = df.iloc[t-200:t]
    X_tr = window.drop(columns=["mort"])
    y_tr = window["mort"]
    X_te = df.iloc[[t]].drop(columns=["mort"])
    y_cv_true.append(df.loc[t, "mort"])

    scaler_cv = StandardScaler()
    X_tr_scaled = scaler_cv.fit_transform(X_tr)
    X_te_scaled = scaler_cv.transform(X_te)

    lasso_model = LassoCV(alphas=lambdas_lasso, cv=tscv, max_iter=5000)
    lasso_model.fit(X_tr_scaled, y_tr)
    lasso_cv_preds.append(lasso_model.predict(X_te_scaled)[0])
    lasso_alphas_selected.append(lasso_model.alpha_)
    lasso_coefs_history.append(lasso_model.coef_)

    ridge_model = RidgeCV(alphas=lambdas_ridge, cv=tscv)
```

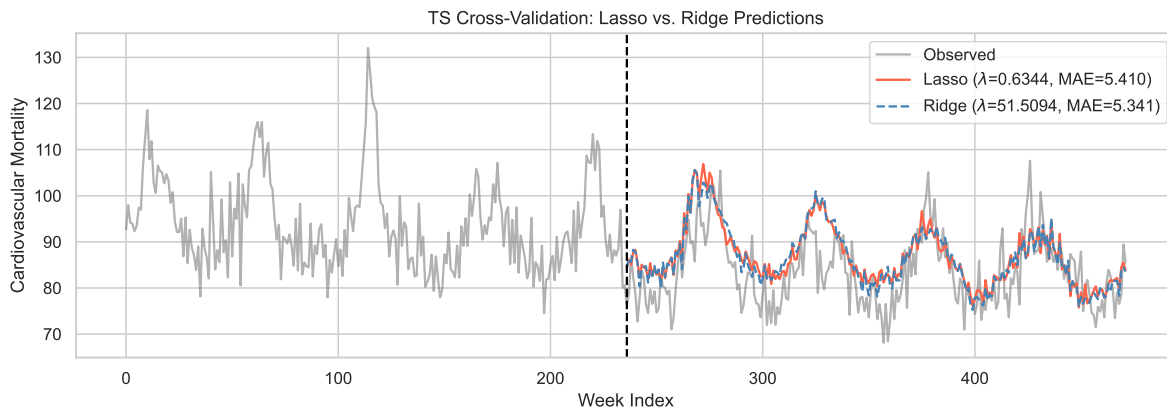
```

ridge_model.fit(X_tr_scaled, y_tr)
ridge_cv_preds.append(ridge_model.predict(X_te_scaled)[0])
ridge_alphas_selected.append(ridge_model.alpha_)

best_lasso_cv_preds = lasso_cv_preds
best_ridge_cv_preds = ridge_cv_preds
best_mae_cv_l = mean_absolute_error(y_cv_true, lasso_cv_preds)
best_mae_cv_r = mean_absolute_error(y_cv_true, ridge_cv_preds)
best_lambda_cv_l = np.median(lasso_alphas_selected)
best_lambda_cv_r = np.median(ridge_alphas_selected)

# Plot Lasso and Ridge CV predictions
fig, ax = plt.subplots(figsize=(11, 4))
sns.lineplot(x=df.index, y=df["mort"], color="grey", alpha=0.6, ax=ax, label="Observed")
sns.lineplot(x=df.index[n_half:], y=best_lasso_cv_preds, color="tomato", ax=ax, label=f"Lasso")
sns.lineplot(x=df.index[n_half:], y=best_ridge_cv_preds, linestyle="--", color="steelblue", ax=ax, label=f"Ridge")
ax.axvline(x=n_half, color="black", linestyle="--")
ax.set_title("TS Cross-Validation: Lasso vs. Ridge Predictions")
ax.set_xlabel("Week Index")
ax.set_ylabel("Cardiovascular Mortality")
ax.legend()
plt.tight_layout()
plt.show()

```



6. (Bonus) Report which lagged features were most frequently selected by the lasso. Because the lasso models will be refit at each time point (in the second half of the data set), you will have to additionally store the lasso solutions along your time series CV pass. Then, look back at the solutions that correspond to the MAE-optimal α value, and choose some way of summarizing which of its components were consistently large in magnitude over time.

```

coefs_history = np.array(lasso_coefs_history) # shape (n_test, p)
selected_freq = np.mean(coefs_history != 0, axis=0)
mean_coefs = np.mean(np.abs(coefs_history), axis=0)

freq_df = pd.DataFrame({
    "Feature": X_train.columns,
    "Selection Frequency": selected_freq,
    "Mean Abs Coefficient": mean_coefs
}).sort_values(by="Selection Frequency", ascending=False)

print("Lasso Feature Selection Summary in Time Series CV:")
print(freq_df.to_string(index=False))

```

Lasso Feature Selection Summary in Time Series CV:

Feature	Selection Frequency	Mean Abs Coefficient
part_lag4	1.000000	2.180398
part_lag8	1.000000	1.466421
temp_lag20	0.978814	0.855876
temp_lag4	0.932203	0.972944
temp_lag24	0.919492	0.842711
temp_lag28	0.783898	0.540944
part_lag12	0.741525	0.433588
part_lag36	0.733051	0.478605
part_lag28	0.427966	0.313738
part_lag20	0.360169	0.298869
temp_lag12	0.211864	0.089081
part_lag32	0.190678	0.034078
temp_lag32	0.165254	0.020817
part_lag16	0.156780	0.062069
temp_lag36	0.156780	0.048660
part_lag24	0.148305	0.078091
temp_lag16	0.131356	0.058523
temp_lag8	0.118644	0.106585

(Type your discussion here)

HP filter

7. (5 pts) Recall in lecture we saw the HP filter could be written explicitly as

$$\hat{\theta} = \underbrace{(I + \lambda D^T D)^{-1}}_K y,$$

where $D \in \mathbb{R}^{(n-2) \times n}$ is the second difference matrix on n points. In other words, defining $K \in \mathbb{R}^{n \times n}$ as above,

$$\hat{\theta}_i = \sum_{j=1}^n K_{ij} y_j, \quad i = 1, \dots, n.$$

Compute the matrix K empirically for a problem of size $n = 100$, and setting the tuning parameter to be $\lambda = 100$; inspect three of its rows, at indices $i = 25, 50, 75$. For each i , plot the i^{th} row as a curve over the underlying position $1, \dots, n$; that is, plot the x-y pairs

$$(x_j, y_j) = (j, K_{ij}), \quad j = 1, \dots, n$$

as a curve. Overlay the curves for all three rows on the same plot, each in a different color. What do these curves look like to you? Use the plot to argue that the HP filter acts like a kernel smoother.

(Type your discussion here)

8. (Bonus) Do a literature search to find theory on the *asymptotically equivalent kernel* for the HP filter. This should have a closed-form. Plot this and comment on whether or not your empirical results adhere to what is known asymptotically.

(Type your discussion here)

Trend filter

9. (Bonus) Implement 5-fold cross-validation in order to tune λ in trend filtering applied to the Boston marathon men's data set. Recall the description of how to set folds in a special "structured" way, for tuning smoothers, given near the end of the lecture notes (weeks 5-6: "Regularization and smoothing"). The code below shows how to run trend filtering and derive estimates at the held-out points for one fold. You can build off this code for your solution. You will need to install the `glmgen` package from GitHub, which you can do using the code that has been commented out.

Important note: just like `glmnet()`, the `trendfilter()` function (in the `glmnet` package) computes its own `lambda` sequence. So you will need to define an initial `lambda` sequence to pass to each subsequent call to `trendfilter()`, so that you can have a consistent grid of tuning parameter values over which to perform cross-validation. We do this below by using the `lambda` sequence that `trendfilter()` itself derived when the trend filtering model is fit on the full data set.

After implementing cross-validation, compute and report the λ value with the smallest cross-validated MAE. Then, lastly, plot the solution at this value of λ when the model is fit to the full data set (this is already available in the `tf` object below.)

```

# Load and preprocess data
boston = pd.read_csv("boston_marathon.csv")
boston = boston[(boston["Year"] >= 1924) & (boston["Event"] == "Men's open division")].copy()

def time_to_minutes(t_str):
    parts = list(map(float, t_str.split(':')))
    return parts[0] * 60 + parts[1] + parts[2] / 60

boston["Minutes"] = boston["Time"].apply(time_to_minutes)
x_b = boston["Year"].values
y_b = boston["Minutes"].values
n_b = len(x_b)

def make_D2(x_grid):
    ng = len(x_grid)
    D = np.zeros((ng - 2, ng))
    for i in range(ng - 2):
        h0 = x_grid[i+1] - x_grid[i]
        h1 = x_grid[i+2] - x_grid[i+1]
        scale = x_grid[i+2] - x_grid[i]
        D[i, i] = 1.0 / (scale * h0)
        D[i, i+1] = - (1.0 / h0 + 1.0 / h1) / scale
        D[i, i+2] = 1.0 / (scale * h1)
    return D

def fit_tf(x_grid, y_grid, lmbda):
    ng = len(x_grid)
    theta = cp.Variable(ng)
    D = make_D2(x_grid)
    obj = cp.Minimize(0.5 * cp.sum_squares(y_grid - theta) + lmbda * cp.norm1(D @ theta))
    prob = cp.Problem(obj)
    prob.solve(solver=cp.CLARABEL)
    return theta.value

# 5-fold cross-validation
k_folds = 5
inds = np.array([i % k_folds for i in range(n_b)]) # rep_len(1:k, n) equivalent
lambdas_tf = np.logspace(-2, 3, 50)

cv_maes_tf = []
for lmbda in lambdas_tf:
    fold_maes = []

```

```

for j in range(k_folds):
    train_mask = inds != j
    test_mask = inds == j

    x_tr, y_tr = x_b[train_mask], y_b[train_mask]
    x_te, y_te = x_b[test_mask], y_b[test_mask]

    # Fit trend filter on training slice
    theta_tr = fit_tf(x_tr, y_tr, lmbda)
    # Evaluate predictions at held-out years using linear interpolation
    yhat = np.interp(x_te, x_tr, theta_tr)

    fold_maes.append(mean_absolute_error(y_te, yhat))
cv_maes_tf.append(np.mean(fold_maes))

best_idx_tf = np.argmin(cv_maes_tf)
best_lambda_tf = lambdas_tf[best_idx_tf]
best_mae_tf = cv_maes_tf[best_idx_tf]

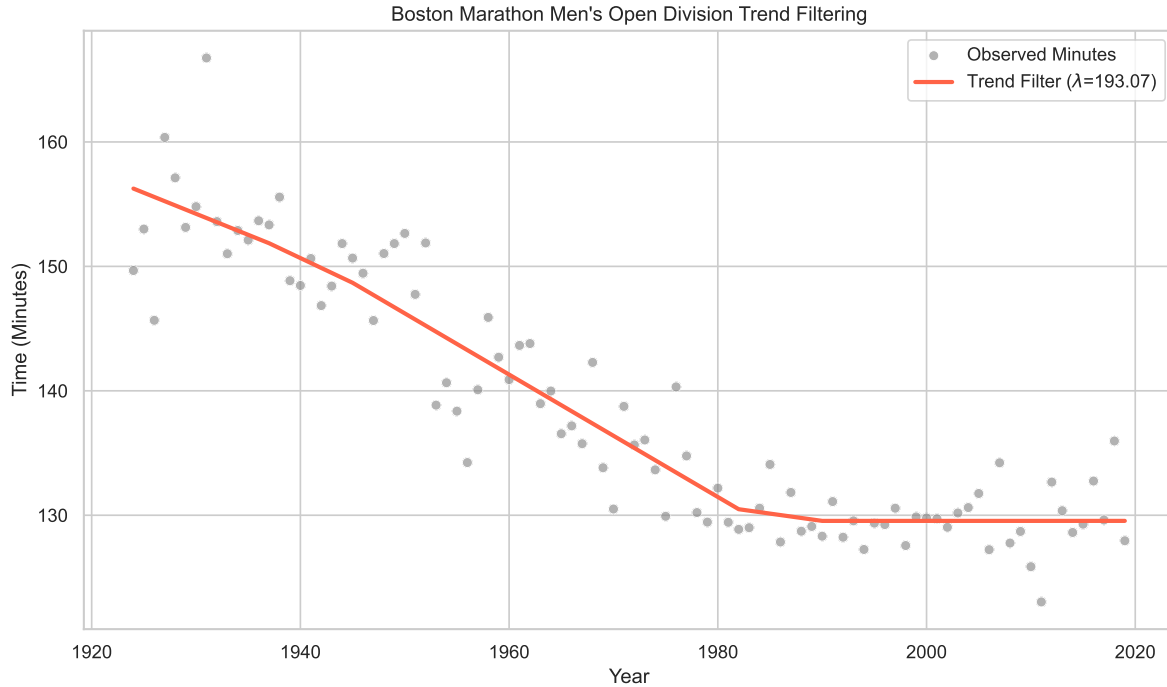
# Fit full model
theta_full = fit_tf(x_b, y_b, best_lambda_tf)

print(f"Optimal Lambda: {best_lambda_tf:.4f}")
print(f"Smallest CV MAE: {best_mae_tf:.4f}")

# Plot solution
fig, ax = plt.subplots(figsize=(10, 6))
sns.scatterplot(x=x_b, y=y_b, color="grey", alpha=0.6, label="Observed Minutes", ax=ax)
sns.lineplot(x=x_b, y=theta_full, color="tomato", lw=2.5, label=f"Trend Filter ( $\lambda$ )", ax=ax)
ax.set_title("Boston Marathon Men's Open Division Trend Filtering")
ax.set_xlabel("Year")
ax.set_ylabel("Time (Minutes)")
ax.legend()
plt.tight_layout()
plt.show()

```

Optimal Lambda: 193.0698
Smallest CV MAE: 2.6415



Spectral analysis

10. (3 pts) Let ω_j , $j = 1, \dots, p$ be fixed and arbitrary frequencies and let U_{j1}, U_{j2} , $j = 1, \dots, p$ be uncorrelated random variables with mean zero, where U_{j1}, U_{j2} have variance σ_j^2 . Define

$$x_t = \sum_{j=1}^p \left(U_{j1} \cos(2\pi\omega_j t) + U_{j2} \sin(2\pi\omega_j t) \right)$$

for $t = 1, 2, 3, \dots$. Prove that this process is stationary, and show that its auto-covariance function is of the form given in lecture.

Proof.

(Type your proof here)

11. (2 pts) Construct a small empirical example to verify the auto-covariance formula you derived in Q10. That is, generate a process with at least $p = 2$ components, compute its auto-correlation function, and compare to the analytic formula you derived.

(Type your discussion here)

Cross-validation

12. (3 pts) Recall the code from lecture that performs time series cross-validation to evaluate the mean absolute error (MAE) of predictions from the linear regression of cardiovascular mortality on 4-week lagged particulate levels. Adapt this to evaluate the MAE of predictions from the regression of cardiovascular mortality on 4-week lagged particulate levels and 4-week lagged temperature (2 features). Fit each regression model using a trailing window of 200 time points (not all past). Plot the predictions, and print the MAE on the plot, following the code from lecture.

Additionally (all drawn on the same figure), plot the fitted values on the training set (burn-in set, times 1 through t_0). The fitted values should come from the initial regression model fit to the burn-in set, plotted in a different color. Print the training MAE on the plot.

```
# Prepare data
df12 = pd.DataFrame({
    "mort": cmort["value"],
    "part4": part["value"].shift(4),
    "temp4": tempr["value"].shift(4)
}).dropna().reset_index(drop=True)

n12 = len(df12)
t0 = 200

# Burn-in fit
fit0_12 = LinearRegression()
fit0_12.fit(df12.iloc[:t0][["part4", "temp4"]], df12.iloc[:t0]["mort"])
train_fitted12 = fit0_12.predict(df12.iloc[:t0][["part4", "temp4"]])
train_mae12 = mean_absolute_error(df12.iloc[:t0]["mort"], train_fitted12)

# TS CV rolling window
preds12 = np.zeros(n12)
preds12[:t0] = np.nan
for t in range(t0, n12):
    window = df12.iloc[t-200:t]
    X_tr = window[["part4", "temp4"]]
    y_tr = window["mort"]
    X_te = df12.iloc[[t]][["part4", "temp4"]]

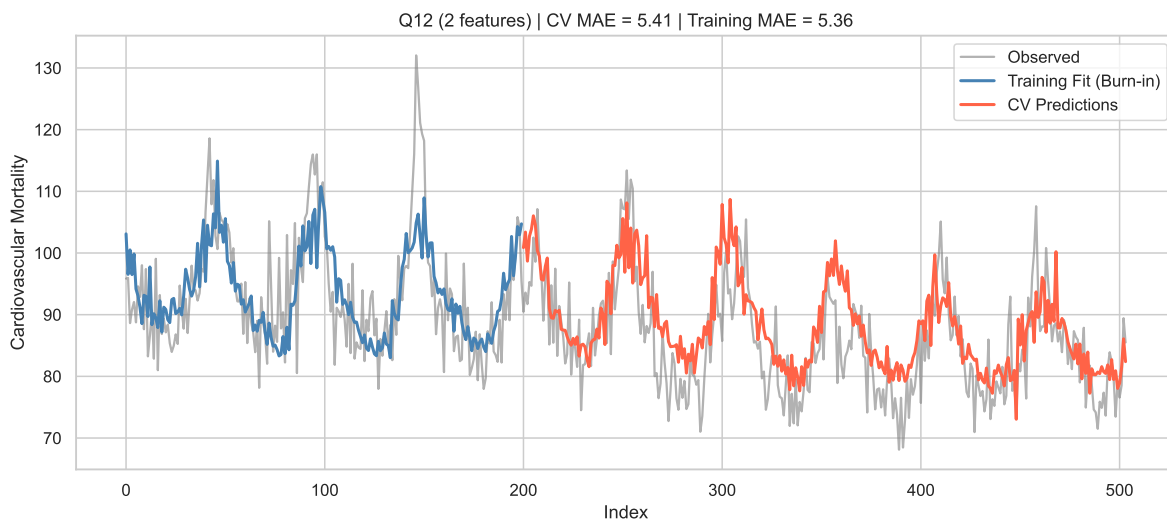
    model = LinearRegression()
    model.fit(X_tr, y_tr)
    preds12[t] = model.predict(X_te)[0]
```

```

cv_mae12 = mean_absolute_error(df12.iloc[t0:]["mort"], preds12[t0:])

fig, ax = plt.subplots(figsize=(11, 5))
sns.lineplot(x=df12.index, y=df12["mort"], color="grey", alpha=0.6, label="Observed", ax=ax)
sns.lineplot(x=df12.index[:t0], y=train_fitted12, color="steelblue", lw=2, label="Training Fit")
sns.lineplot(x=df12.index[t0:], y=preds12[t0:], color="tomato", lw=2, label="CV Predictions")
ax.set_title(f"Q12 (2 features) | CV MAE = {cv_mae12:.2f} | Training MAE = {train_mae12:.2f}")
ax.set_xlabel("Index")
ax.set_ylabel("Cardiovascular Mortality")
ax.legend()
plt.tight_layout()
plt.show()

```



13. (2 pts) Repeat the same exercise as in Q12 but now with multiple lags per variable: use lags 4, 8, 12 for each of particulate level and temperature (thus 6 features in total). Did the training MAE go down? Did the cross-validated MAE go down? Discuss.

```

df13 = pd.DataFrame({
    "mort": cmort["value"],
    "part4": part["value"].shift(4),
    "part8": part["value"].shift(8),
    "part12": part["value"].shift(12),
    "temp4": tempr["value"].shift(4),
    "temp8": tempr["value"].shift(8),
    "temp12": tempr["value"].shift(12)
}).dropna().reset_index(drop=True)

```

```

n13 = len(df13)
features13 = ["part4", "part8", "part12", "temp4", "temp8", "temp12"]

# Burn-in fit
fit0_13 = LinearRegression()
fit0_13.fit(df13.iloc[:t0][features13], df13.iloc[:t0]["mort"])
train_fitted13 = fit0_13.predict(df13.iloc[:t0][features13])
train_mae13 = mean_absolute_error(df13.iloc[:t0]["mort"], train_fitted13)

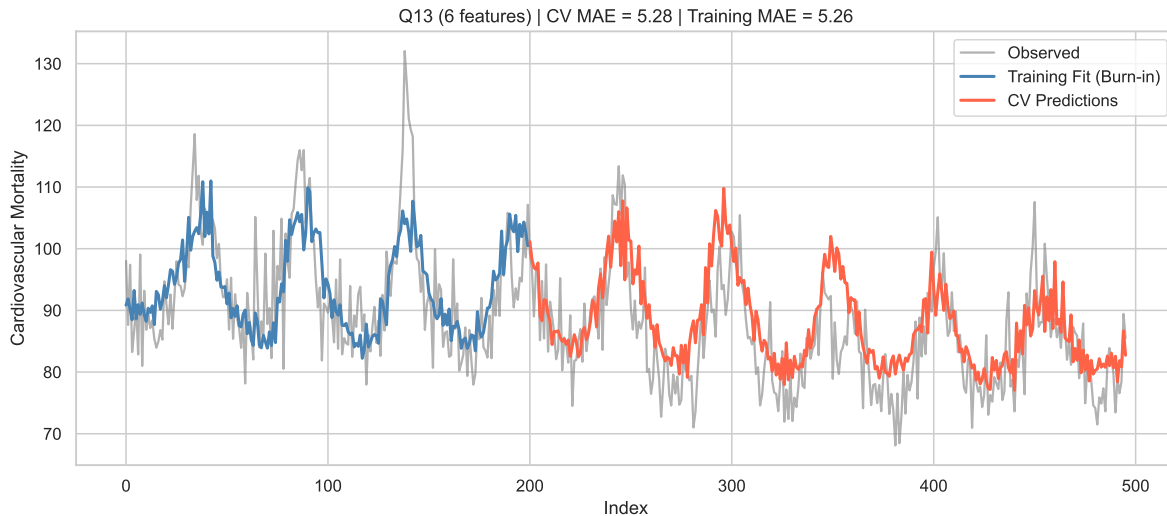
# TS CV rolling window
preds13 = np.zeros(n13)
preds13[:t0] = np.nan
for t in range(t0, n13):
    window = df13.iloc[t-200:t]
    X_tr = window[features13]
    y_tr = window["mort"]
    X_te = df13.iloc[[t]][features13]

    model = LinearRegression()
    model.fit(X_tr, y_tr)
    preds13[t] = model.predict(X_te)[0]

cv_mae13 = mean_absolute_error(df13.iloc[t0:]["mort"], preds13[t0:])

fig, ax = plt.subplots(figsize=(11, 5))
sns.lineplot(x=df13.index, y=df13["mort"], color="grey", alpha=0.6, label="Observed", ax=ax)
sns.lineplot(x=df13.index[:t0], y=train_fitted13, color="steelblue", lw=2, label="Training Fitted")
sns.lineplot(x=df13.index[t0:], y=preds13[t0:], color="tomato", lw=2, label="CV Predictions")
ax.set_title(f"Q13 (6 features) | CV MAE = {cv_mae13:.2f} | Training MAE = {train_mae13:.2f}")
ax.set_xlabel("Index")
ax.set_ylabel("Cardiovascular Mortality")
ax.legend()
plt.tight_layout()
plt.show()

```



(Type your discussion here)

14. (2 pts) Repeat once more the same exercise as in the last question but now with many lags per variable: use lags 4, 5, ..., through 50 for each of particulate level and temperature (thus $47 \times 2 = 94$ features in total). Did the training MAE go down? Did the cross-validated MAE go down? Are you surprised? Discuss.

```
lag_seq = range(4, 51)
df_dict = {"mort": cmort["value"]}
for k in lag_seq:
    df_dict[f"part{k}"] = part["value"].shift(k)
    df_dict[f"temp{k}"] = tempr["value"].shift(k)

df14 = pd.DataFrame(df_dict).dropna().reset_index(drop=True)
n14 = len(df14)
features14 = [f"part{k}" for k in lag_seq] + [f"temp{k}" for k in lag_seq]

# Burn-in fit
fit0_14 = LinearRegression()
fit0_14.fit(df14.iloc[:t0][features14], df14.iloc[:t0]["mort"])
train_fitted14 = fit0_14.predict(df14.iloc[:t0][features14])
train_mae14 = mean_absolute_error(df14.iloc[:t0]["mort"], train_fitted14)

# TS CV rolling window
preds14 = np.zeros(n14)
preds14[:t0] = np.nan
for t in range(t0, n14):
```



```

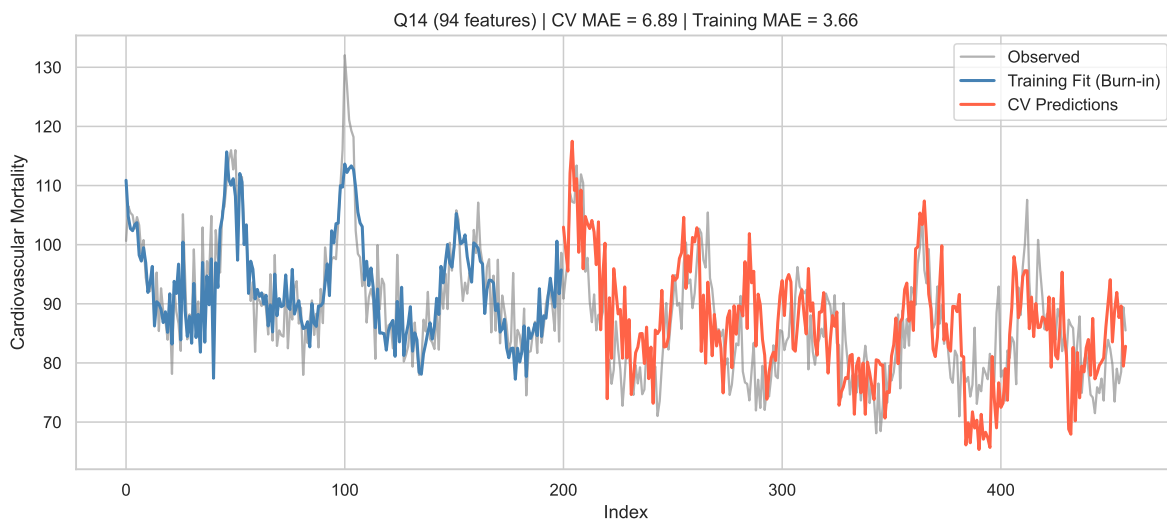
window = df14.iloc[t-200:t]
X_tr = window[features14]
y_tr = window["mort"]
X_te = df14.iloc[[t]][features14]

model = LinearRegression()
model.fit(X_tr, y_tr)
preds14[t] = model.predict(X_te)[0]

cv_mae14 = mean_absolute_error(df14.iloc[t0:]["mort"], preds14[t0:])

fig, ax = plt.subplots(figsize=(11, 5))
sns.lineplot(x=df14.index, y=df14["mort"], color="grey", alpha=0.6, label="Observed", ax=ax)
sns.lineplot(x=df14.index[:t0], y=train_fitted14, color="steelblue", lw=2, label="Training Fit (Burn-in)", ax=ax)
sns.lineplot(x=df14.index[t0:], y=preds14[t0:], color="tomato", lw=2, label="CV Predictions", ax=ax)
ax.set_title(f"Q14 (94 features) | CV MAE = {cv_mae14:.2f} | Training MAE = {train_mae14:.2f}")
ax.set_xlabel("Index")
ax.set_ylabel("Cardiovascular Mortality")
ax.legend()
plt.tight_layout()
plt.show()

```



(Type your discussion here)

More features, the merrier?

15. (2 pts) Let y_i be an arbitrary response, and $x_i \in \mathbb{R}^p$ be an arbitrary feature vector, for $i = 1, \dots, n$. Let

$$\tilde{x}_i = (x_{i1}, \dots, x_{ip}, \tilde{x}_{i,p+1}), \quad i = 1, \dots, n$$

be the result of appending one more feature. Let $\hat{y}_i$ denote the fitted values from the regression of y_i on x_i , and let $\tilde{y}_i$ denote the fitted values from the regression of y_i on $\tilde{x}_i$. Prove that

$$\sum_{i=1}^n (y_i - \tilde{y}_i)^2 \leq \sum_{i=1}^n (y_i - \hat{y}_i)^2.$$

In other words, *the training MSE will never get worse as we add features* to a given sample regression problem.

Proof.

(Type your proof here)

16. (2 pts) How many linearly independent features do we need (how large should p be) in order to achieve a perfect training accuracy, i.e., training MSE of zero? Why?

Answer.

(Type your answer here)

17. (Bonus) Implement an example in Python in order to verify your answer to Q16 empirically. Use the cardiovascular mortality data, using enough lagged features. You should be able to plot the fitted values from the training set and see that they match the observations perfectly.

```
# Create p = 200 features (lags 4 through 103 for part and temp)
lags_q17 = range(4, 104)
df17_dict = {"mort": cmort["value"]}
for k in lags_q17:
    df17_dict[f"part{k}"] = part["value"].shift(k)
    df17_dict[f"temp{k}"] = tempr["value"].shift(k)

df17 = pd.DataFrame(df17_dict).dropna().reset_index(drop=True)
features17 = [f"part{k}" for k in lags_q17] + [f"temp{k}" for k in lags_q17]

# Fit on first t0 = 200 points
fit_q17 = LinearRegression()
```

```

fit_q17.fit(df17.iloc[:t0][features17], df17.iloc[:t0]["mort"])
train_fitted17 = fit_q17.predict(df17.iloc[:t0][features17])
train_mae17 = mean_absolute_error(df17.iloc[:t0]["mort"], train_fitted17)

print(f"Training MAE with p=200 features and n=200 observations: {train_mae17:.8e}")

# Plot perfect training fit
fig, ax = plt.subplots(figsize=(10, 5))
sns.lineplot(x=df17.index[:t0], y=df17.iloc[:t0]["mort"], color="grey", marker="o", linestyle="solid")
sns.lineplot(x=df17.index[:t0], y=train_fitted17, color="purple", marker="x", linestyle="dotted",)
ax.set_title(f"Q17: Perfect Fit on Training Set (Training MAE = {train_mae17:.2e})")
ax.set_xlabel("Week Index")
ax.set_ylabel("Cardiovascular Mortality")
ax.legend()
plt.tight_layout()
plt.show()

```

Training MAE with p=200 features and n=200 observations: 1.17523768e-13

